

Quatro Áreas da Computação Visual

síntese · **processamento** · **visão** · **visualização**

Para cada área, uma aplicação de repositório público foi executada e medida. Todos os números e figuras deste relatório vêm dessa execução — nada foi copiado de documentação.

Ambiente Python 3.12 · Windows 11 · Intel Iris Xe

Contexto gráfico OpenGL 3.3 Core, offscreen

Execução completa 22 s

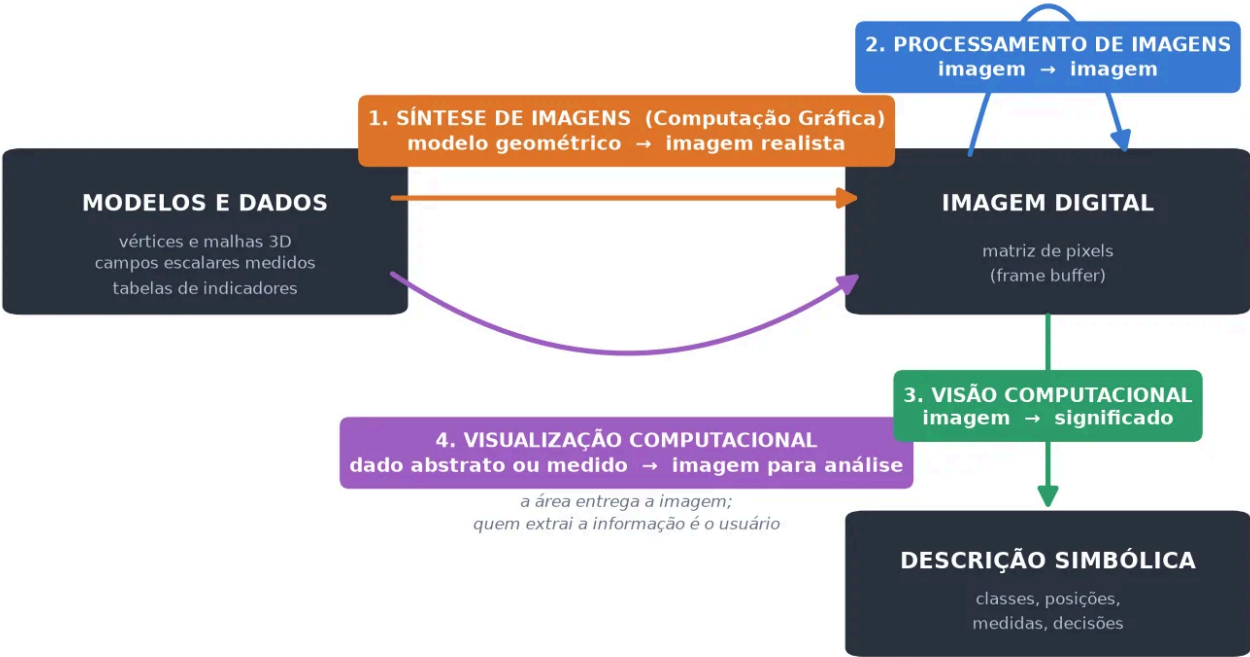
Versão online claude.ai/code/artifact/e7cf0146

Código e figuras AC1/

A tese: cada área se define pelo par (o que entra, o que sai)

O material da disciplina descreve as quatro áreas como “*áreas de pesquisa com identidade própria e focos distintos, mas que interagem entre si e compartilham técnicas e algoritmos*”. O que dá identidade a cada uma não é a técnica — várias são compartilhadas — e sim **o tipo de dado que entra e o tipo de dado que sai**.

Relacionamento entre as áreas da Computação Visual



As caixas são TIPOS DE DADO; as setas são as ÁREAS. Cada área se define pelo par (o que entra, o que sai). São áreas com identidade própria e focos distintos, mas que interagem: a Visualização usa o motor de síntese da área 1, e a Visão Computacional usa o Processamento de Imagens como etapa de pré-processamento.

As caixas são tipos de dado; as setas são as áreas. Essa leitura separa as quatro sem ambiguidade. Figura gerada por `00_comparativo/gerar_comparativo.py`.

AS QUATRO ÁREAS EM UMA LINHA CADA

#	ÁREA	ENTRA	SAI	PERGUNTA QUE RESPONDE
1	Síntese de Imagens (Computação Gráfica)	modelo geométrico, material, luz, câmera	imagem matricial	Como este objeto se pareceria?
2	Processamento de Imagens	imagem	imagem	Como deixar esta imagem melhor ou mais útil?
3	Visão Computacional (Visão Artificial)	imagem	descrição simbólica	O que há nesta imagem?
4	Visualização Computacional	dados medidos ou abstratos	imagem para análise	O que estes dados estão dizendo?

Repare na simetria: **Síntese** e **Visão** são operações inversas — dados → imagem contra imagem → dados. O **Processamento** é a única que permanece no mesmo domínio. E a **Visualização** compartilha o motor de renderização da Síntese, mas troca o objetivo de *realismo* por *legibilidade*.

As quatro áreas da Computação Visual — um resultado executado de cada uma

1. SÍNTESE DE IMAGENS (Computação Gráfica)



ENTRADA: modelo geométrico
19.895 vértices / 37.986 triângulos
SAÍDA: imagem 900x900
rasterizada pela GPU

2. PROCESSAMENTO DE IMAGENS



ENTRADA: imagem 512x480x3
(fotografia)
SAÍDA: imagem 512x480
(mesma natureza de dado)

3. VISÃO COMPUTACIONAL (Visão Artificial)



ENTRADA: imagem 810x1080x3
(2,6 MB de pixels)
SAÍDA: 6 objetos nomeados
(974 bytes de JSON)

4. VISUALIZAÇÃO COMPUTACIONAL



ENTRADA: campo escalar 3D
10.368.384 voxels (9,9 MB)
SAÍDA: imagem para ANÁLISE
(ray casting / DVR)

Um resultado executado de cada área, com a entrada e a saída anotadas sob cada painel.

Como executar

TERMINAL

```
# ambiente
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install torch --index-url https://download.pytorch.org/whl/cpu

# roda as quatro áreas em sequência e grava relatorio/log_execucao.txt
python executar_tudo.py
```

Sobre a versão do OpenCV. É obrigatório usar a série **4.x**: o OpenCV 5.0 removeu `cv2.HOGDescriptor`, usado na comparação entre abordagem clássica e aprendizado profundo da Área 3. O `requirements.txt` fixa `opencv-python==4.14.0.94`.

1 Síntese de Imagens

Computação Gráfica · ModernGL sobre OpenGL 3.3 Core

modelo geométrico → imagem matricial

Técnicas para gerar representações visuais a partir de especificações geométricas e de atributos visuais dos seus componentes. Modelagem e rendering. Objetivo: “mundo” 3D no computador.

aula02/intro_computacao_visual.pdf, slide 15

REPOSITÓRIO	github.com/moderngl/moderngl
MODELO	aula08/objetos/dragao.obj — material da própria disciplina
PROGRAMA	01_sintese_imagens/sintese_opengl.py

A escolha é deliberada: o material da aula 02 dedica a segunda metade ao **OpenGL moderna (3.3+)** e ao pipeline vértices → montagem de polígonos → rasterização → *fragment shader* → *frame buffer*. O programa percorre exatamente esse caminho, com cada etapa isolada.

ASPECTOS DA ÁREA EVIDENCIADOS

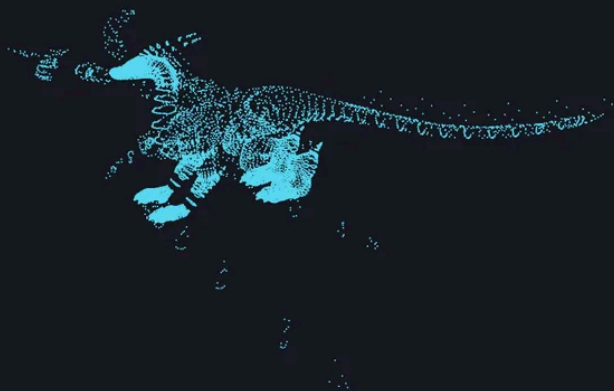
1. **Modelagem** — leitura da malha Wavefront `.obj`: só vértices e faces, nenhum pixel.
2. **Poligonização** — normais por vértice calculadas como média das normais das faces incidentes.
3. **Transformações geométricas** — as matrizes `Model`, `View` e `Projection` são construídas à mão em coordenadas homogêneas 4×4, não importadas de biblioteca.
4. **Rasterização** — a conversão vetorial → matricial acontece na GPU.
5. **Modelo de iluminação** — Phong (ambiente + difusa + especular) escrito em GLSL, com correção gama.
6. **Remoção de superfícies ocultas** — *z-buffer*.
7. **Frame buffer** — leitura do buffer de cor com MSAA 4× e gravação em PNG.

Há ainda um **ajuste de enquadramento** implementado à parte: o programa projeta todos os vértices, mede a caixa envolvente já em coordenadas normalizadas de dispositivo e aplica escala + translação 2D no espaço de recorte. Como $x_{ndc} = x_{clip} / w$, multiplicar `x_clip` por `k` e somar `-k·cx·w` equivale a $(x_{ndc} - cx) \cdot k$ — sem tocar em `z`, o *z-buffer* continua válido.

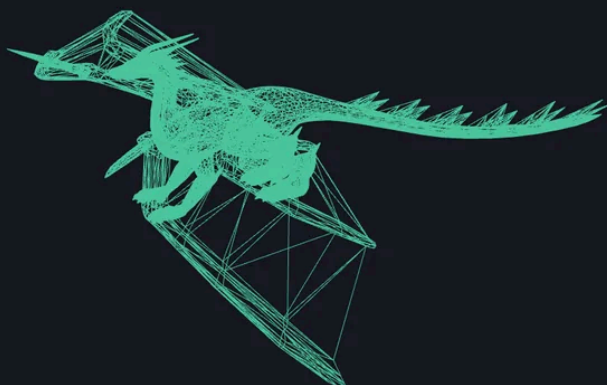
SAÍDA DA EXECUÇÃO – SINTESE_OPENGL.PY

```
[1] MODELAGEM - arquivo .....: dragao.obj (2391 KB)
    vertices .....: 19,895
[2] POLIGONIZACAO - triangulos .: 37,986
    GL_VERSION .....: 3.3.0 - Build 32.0.101.7084
    GL_RENDERER .....: Intel(R) Iris(R) Xe Graphics
    VBO enviado a GPU .....: 466 KB
    ajuste de enquadramento ....: fator 2.05x em coordenadas de recorte
[4-7] 4. Shading de Phong (amb + dif + esp) 0.9 ms -> saida/d_phong.png
```

1. Nuvem de vértices (modelo geométrico)



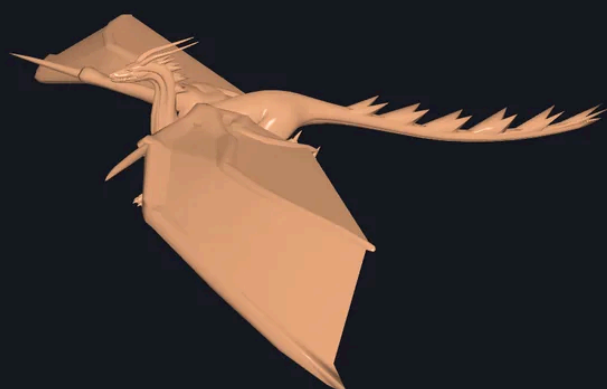
2. Malha de triângulos (wireframe)



3. Rasterização + z-buffer (sem luz)



4. Shading de Phong (amb + dif + esp)



Os quatro quadros são **o mesmo modelo, na mesma câmera**, variando apenas o estágio do pipeline. A “forma” já existe no quadro 1 — ela é dado de entrada. O que os estágios seguintes acrescentam é **aparência**.

BALANÇO DA ÁREA 1

- **Entrada:** **19.895** vértices e **37.986** triângulos — 466 KB de números, nenhum pixel.
- **Saída:** imagem $900 \times 900 = \mathbf{810.000}$ pixels, em **0,9 ms**.
- Nenhuma imagem foi lida como entrada. A imagem foi **sintetizada**.
- O custo é dominado pela rasterização, e não pela leitura do modelo — daí a área ser inseparável do hardware gráfico.

2

Processamento de Imagens

OpenCV 4.14 · imagens de teste do próprio repositório

imagem

→

imagem

Técnicas de transformação de imagens descritas como “matrizes” de pixels. Objetivo: melhorar características visuais (aumentar contraste, melhorar foco, reduzir ruído, eliminar distorções); extrair elementos de interesse; ou mesmo “transformar” a imagem, criando efeitos visuais.

aula02/intro_computacao_visual.pdf, slide 23

REPOSITÓRIO github.com/opencv/opencv

IMAGENS `samples/data/fruits.jpg` e `samples/data/smarties.png`, do próprio repositório do OpenCV

PROGRAMA `02_processamento_imagens/processamento_opencv.py`

Os blocos seguem a estrutura do material `files/PI.pdf`, que organiza a área em aquisição → restauração/realce → segmentação → extração de atributos → classificação.

A · Aquisição: resolução espacial e gradação tonal

Reprodução das figuras 3.3 e 3.4 do `PI.pdf`: o efeito da **amostragem** (reduzir a grade de 512×480 para 64×60 e 16×15) e da **quantização** (de 256 para 16, 4 e 2 níveis por canal). São os dois eixos independentes que definem quanta informação uma imagem digital carrega.

(A) Aquisição: efeito da resolução espacial e da quantização



Amostragem e quantização degradam a imagem por mecanismos diferentes: uma perde **detalhe espacial**, a outra perde **gradação tonal**.

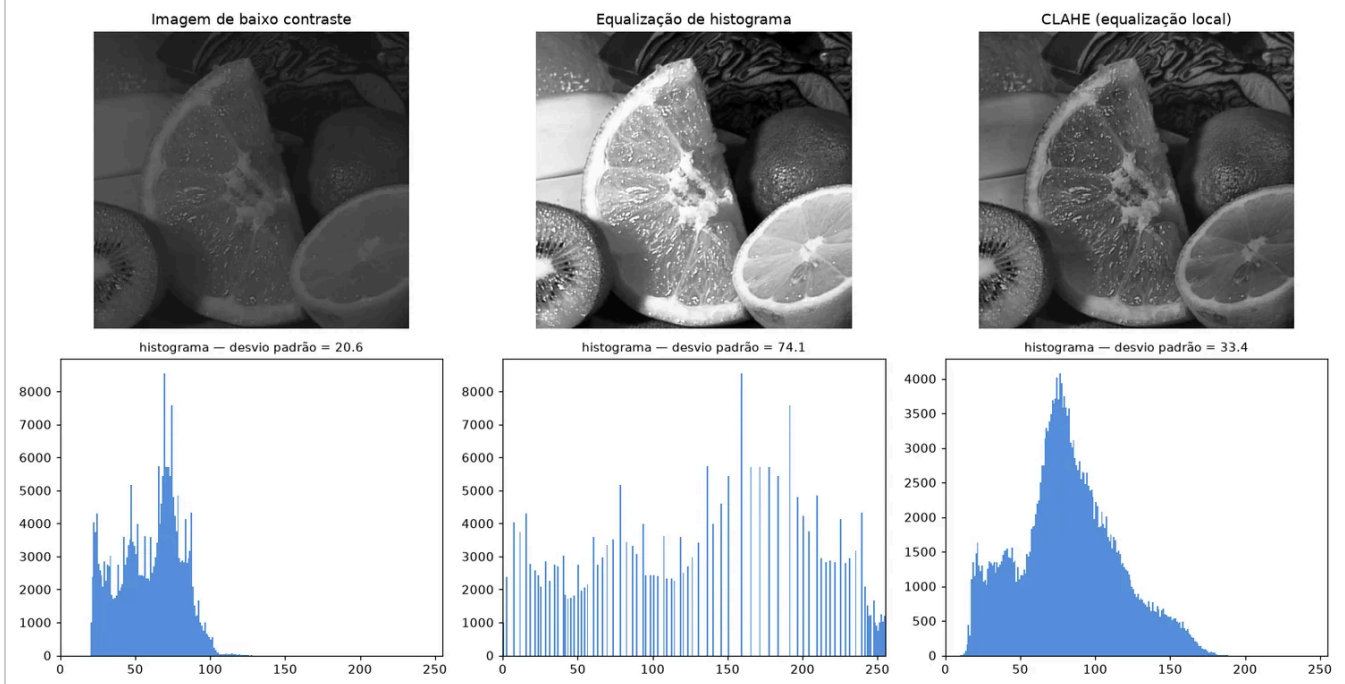
B · Realce: o histograma como descritor global

EFEITO DA EQUALIZAÇÃO SOBRE A DISTRIBUIÇÃO DE INTENSIDADES

IMAGEM	MÉDIA	DESVIO PADRÃO	FAIXA
Tons de cinza (original)	88,7	45,8	[0, 238]
Baixo contraste (simulado)	59,4	20,6	[20, 127]
Equalização de histograma	129,0	74,1	[0, 255]
CLAHE (equalização local)	81,1	33,4	[10, 188]

A equalização **triplicou o desvio padrão** (20,6 → 74,1) e espalhou os valores por toda a faixa. O ponto conceitual: a operação não sabe nada sobre o conteúdo — ela apenas redistribui a função de distribuição acumulada dos níveis de cinza.

(B) Realce: o histograma como descritor global da imagem



O histograma é o descritor que orienta a decisão — e é também o que revela por que a equalização global amplia ruído onde o CLAHE não amplia.

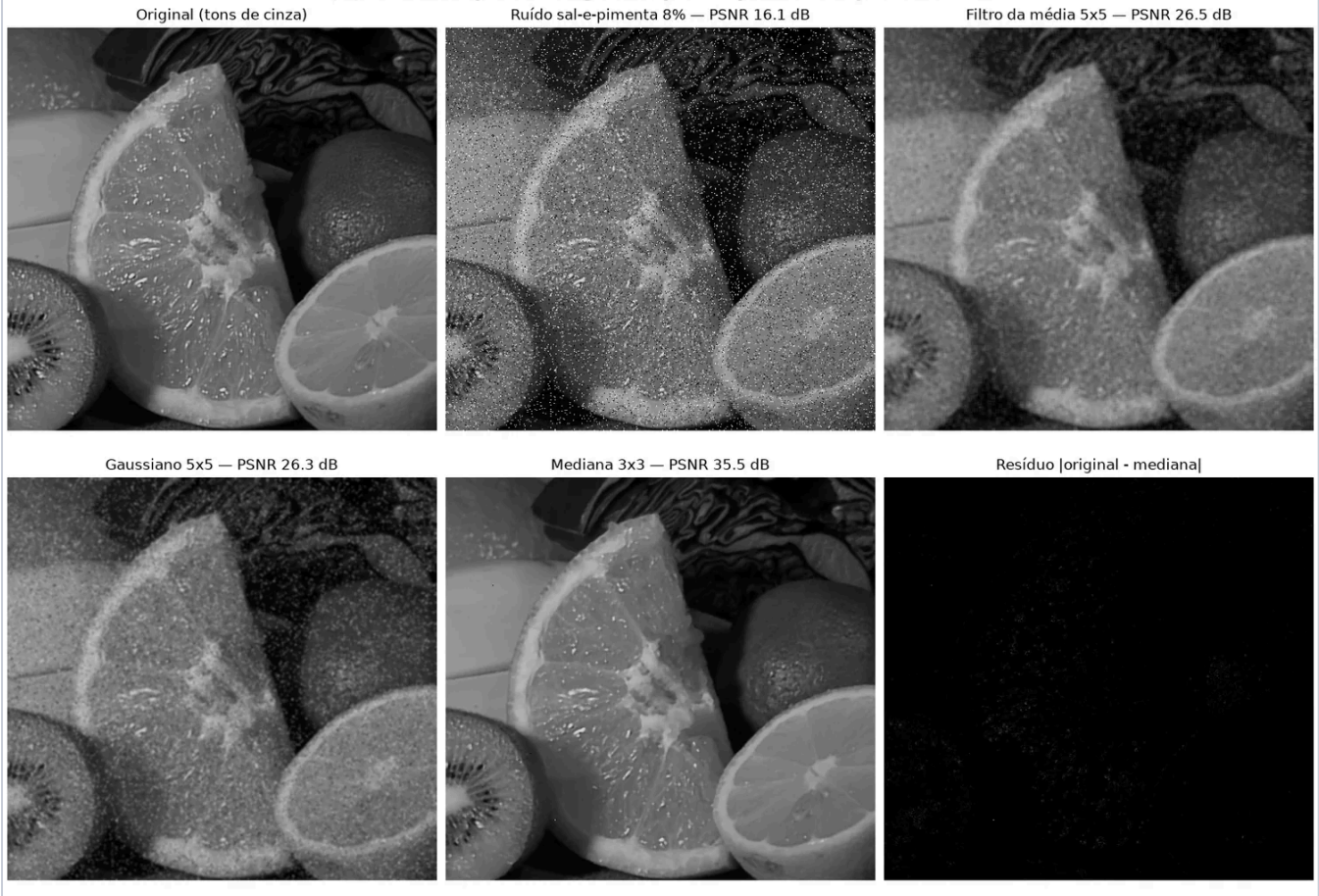
C • Restauração: ruído e filtragem

PSNR CONTRA A ORIGINAL, COM 8 % DOS PIXELS CORROMPIDOS POR SAL-E-PIMENTA

FILTRO	PSNR	NATUREZA
imagem ruidosa (sem filtro)	16,11 dB	—
média 5×5	26,49 dB	linear
gaussiano 5×5	26,26 dB	linear
mediana 3×3	35,51 dB	não-linear

A mediana ganha por quase **9 dB** usando uma janela *menor* que a dos concorrentes. O motivo é estrutural: a mediana de um conjunto é imune a valores extremos, enquanto qualquer média ponderada espalha o pixel corrompido pela vizinhança.

(C) Restauração: filtragem espacial para remoção de ruído



O último painel mostra o **resíduo** |original – mediana|: onde a filtragem agiu. Ele é quase todo pontos isolados — o filtro removeu o ruído sem destruir as bordas.

D • Realce de bordas: convolução

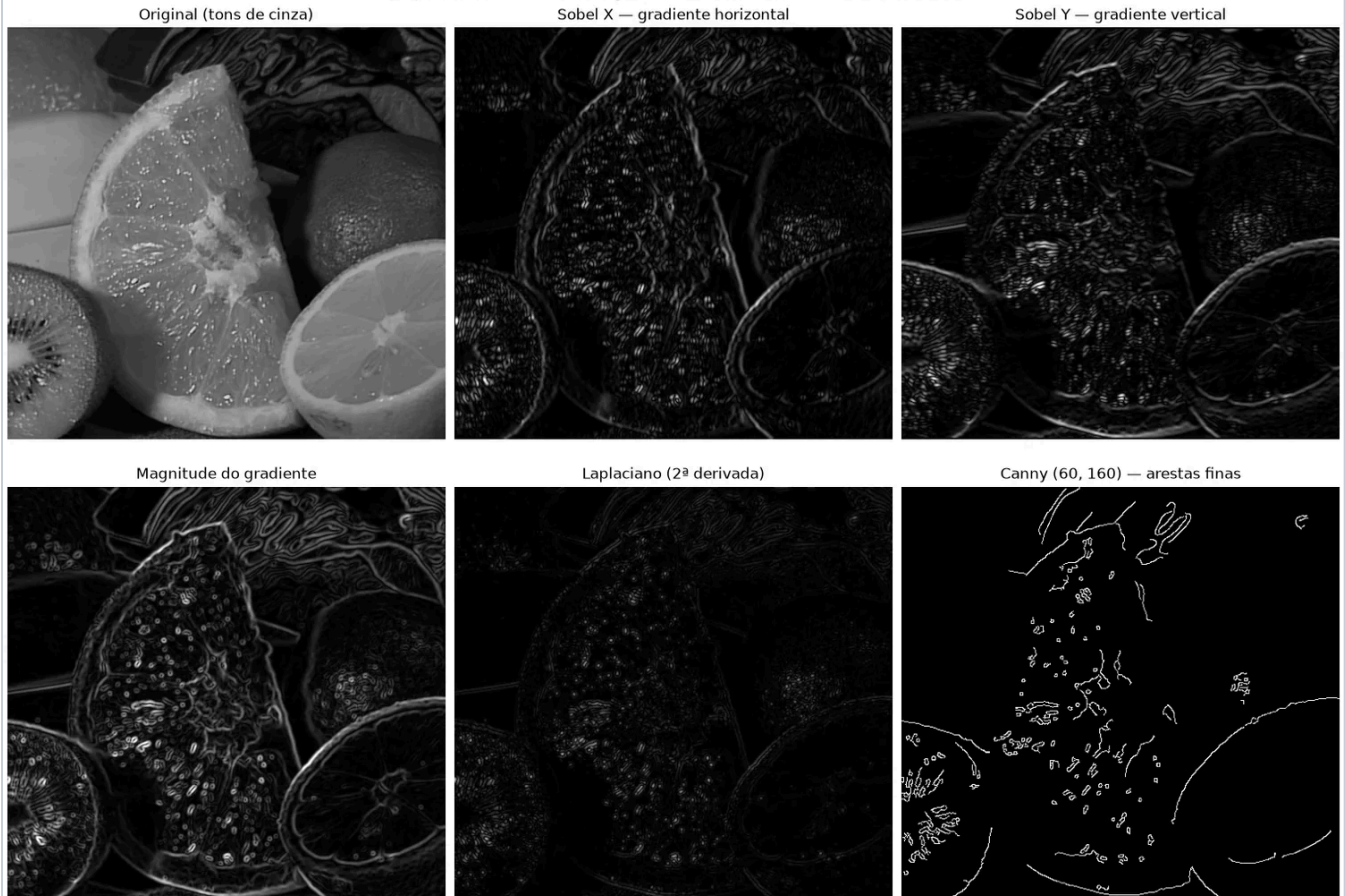
MÁSCARAS APLICADAS NA VIZINHANÇA 3×3

```
Sobel_x = [[-1 0 1]
            [-2 0 2]
            [-1 0 1]]
Sobel_y = [[-1 -2 -1]
            [ 0  0  0]
            [ 1  2  1]]
```

pixels de borda detectados pelo Canny: **6,057** (2.46% da imagem)

É a operação que mais aproxima esta área da seguinte: reduzir a imagem a suas descontinuidades é o primeiro passo de quase todo sistema de visão.

(D) Realce: convolução com máscaras derivativas



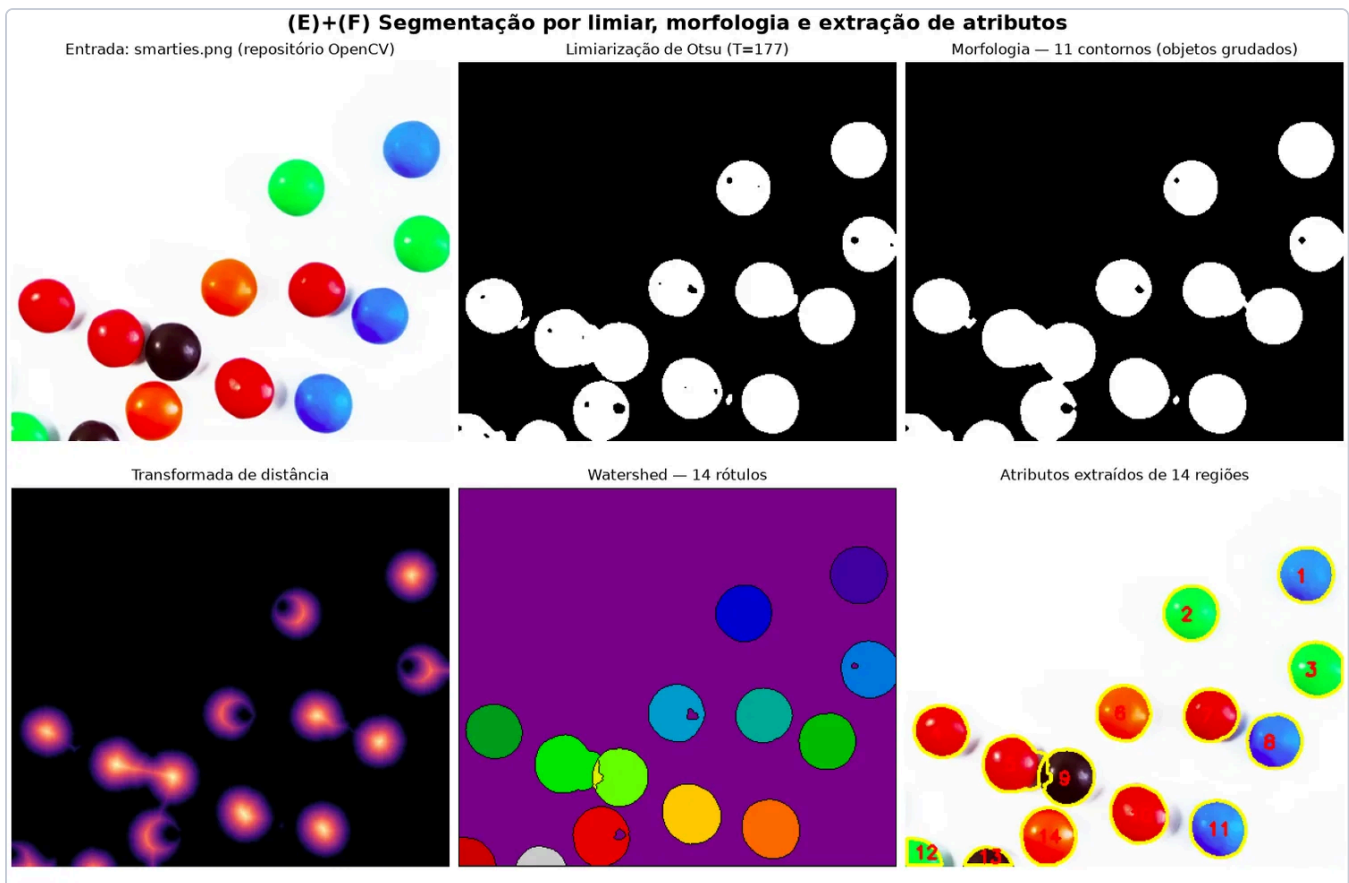
Da primeira derivada direcional (Sobel X e Y) à magnitude do gradiente, à segunda derivada (Laplaciano) e às arestas finas do Canny.

E • Segmentação e extração de atributos

A limiarização só faz sentido quando o histograma é **bimodal**, como ensina o [PI.pdf](#). Por isso esta etapa troca de imagem, passando para [smarties.png](#).

- Limiar escolhido **automaticamente pelo método de Otsu**: $T = 177$
- Após a morfologia (abertura + fechamento): **11 contornos** — objetos encostados saem grudados
- Após transformada de distância + **watershed**: **15 marcadores**, **14 regiões** acima de 300 px

Atributos extraídos de cada região: área, perímetro e circularidade $4\pi A/P^2$. As regiões inteiras convergem para circularidade $\approx 0,89$ (o valor teórico de um círculo é 1,0; a discretização em pixels custa o resto). As duas regiões cortadas pela borda da imagem caem para **0,73** e **0,59** — o próprio atributo denuncia que o objeto está incompleto.



A transformada de distância transforma “onde há objeto” em “quão fundo dentro do objeto” — e é esse máximo local que dá ao watershed um marcador por confeito, separando os que a limiarização havia fundido.

BALANÇO DA ÁREA 2

- **Entrada:** imagem matricial $512 \times 480 \times 3$. **Saída:** imagem matricial 512×480 — a mesma natureza de dado.
- A informação é sempre reorganizada **dentro da própria grade de pixels**.
- O sistema não sabe que são frutas ou confeitos: conhece intensidades e vizinhanças. Área, perímetro e circularidade são **atributos geométricos**, não semânticos.
- Dar nome ao objeto é o trabalho da área seguinte.

3

Visão Computacional

Visão Artificial · Ultralytics YOLOv8n, pesos COCO

imagem

→

descrição simbólica

Colocar “o sentido” da visão na máquina. Problema extremamente complexo — visão requer inteligência! Problema mais delimitado: reconhecimento de padrões em imagens. p.ex.

<https://pjreddie.com/darknet/yolo/>

[aula02/intro_computacao_visual.pdf](#), slide 28

A aplicação escolhida é o **próprio exemplo citado pelo slide**.

REPOSITÓRIO	github.com/ultralytics/ultralytics
PESOS	<code>yolov8n.pt</code> — 3.157.200 parâmetros, 80 classes do conjunto COCO
IMAGEM	<code>bus.jpg</code> , de github.com/ultralytics/assets
PROGRAMA	<code>03_visao_computacional/visao_yolo.py</code>

O programa é organizado segundo os **seis passos do “típico sistema de visão”** apresentados nos slides 31 a 45.

Passos 1 e 2 · Aquisição e pré-processamento

SAÍDA DA EXECUÇÃO — VISAO_YOLO.PY

PASSO 1 — AQUISIÇÃO

dimensões: 810x1080x3 = 2,624,400 bytes brutos

PASSO 2 — PRÉ-PROCESSAMENTO

letterbox: 810x1080 -> 640x640 (escala 0.593, borda dx=80 dy=0)

BGR -> RGB, HWC -> NCHW: tensor (1, 3, 640, 640)

normalização: uint8 [0,255] -> float32 [0,1]

Todas as operações do Passo 2 — redimensionamento com preservação de proporção, preenchimento de borda, conversão de espaço de cor, normalização — **são Processamento de Imagens**. É aqui que a Área 2 entra como serviço da Área 3.

Passos 3 e 5 · Processamento e extração de características

MAPAS CAPTURADOS POR GANCHOS NOS BLOCOS INICIAIS DA REDE

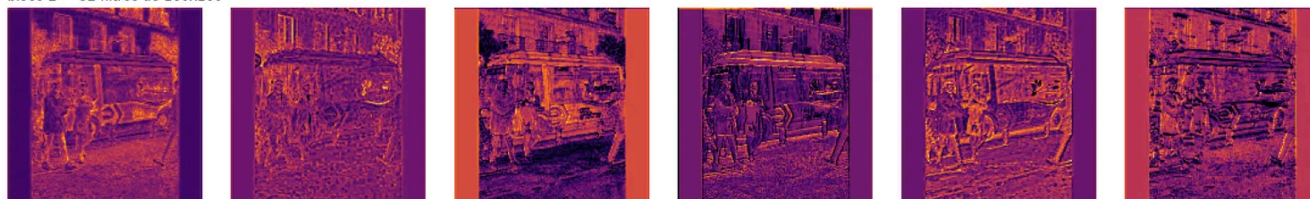
BLOCO	FORMATO DO MAPA	INTERPRETAÇÃO
0	(1, 16, 320, 320)	16 filtros de 320×320 — bordas e gradientes
2	(1, 32, 160, 160)	32 filtros de 160×160 — texturas e contornos
4	(1, 64, 80, 80)	64 filtros de 80×80 — partes de objeto
6	(1, 128, 40, 40)	128 filtros de 40×40 — padrões semânticos

**PASSO 5 — Extração de características: mapas aprendidos pela rede
(bordas e texturas nas camadas rasas; padrões semânticos nas profundas)**

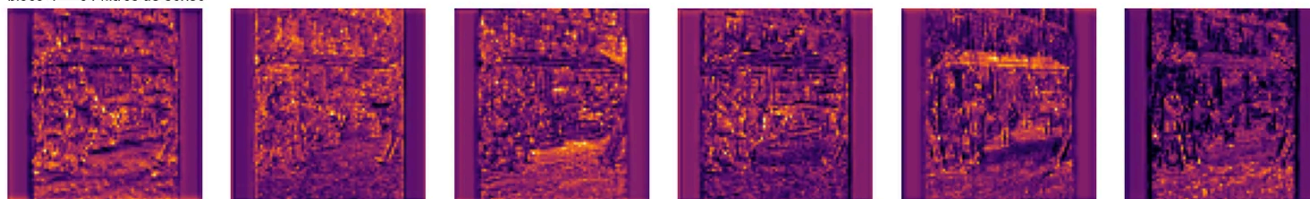
bloco 0 — 16 filtros de 320x320



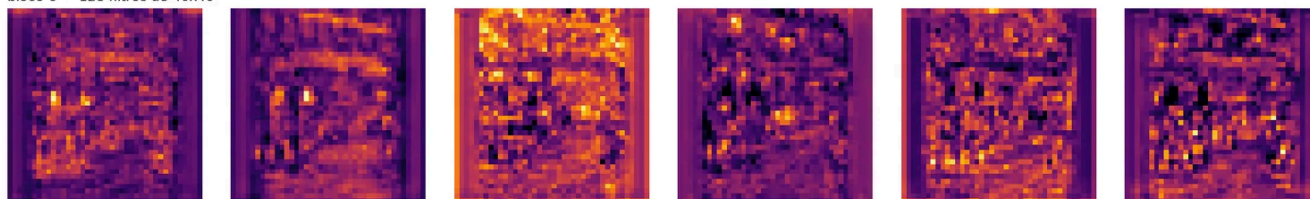
bloco 2 — 32 filtros de 160x160



bloco 4 — 64 filtros de 80x80



bloco 6 — 128 filtros de 40x40



Esta figura é o coração da diferença entre as áreas 2 e 3. Os mapas do bloco 0 são **visualmente indistinguíveis de uma saída de Sobel** — a rede redescobriu, sozinha, detectores de borda. Conforme se desce, a resolução espacial cai e o número de canais sobe: a representação deixa de ser “uma imagem” e passa a ser “um vetor de evidências”.

Passos 4 e 6 · Análise e reconhecimento

SAÍDA DA ÁREA — DESCRIÇÃO SIMBÓLICA DA CENA (NMS COM IOU 0,45; CONFIANÇA MÍNIMA 0,25)

#	CLASSE	CONFIANÇA	CAIXA (X1, Y1, X2, Y2)	ÁREA (PX)
1	bus	0,873	23, 231, 805, 757	411.059
2	person	0,866	49, 399, 245, 903	99.214
3	person	0,853	670, 392, 810, 877	67.999
4	person	0,825	222, 406, 345, 858	55.769
5	person	0,261	0, 550, 63, 873	20.346
6	stop sign	0,255	0, 254, 33, 325	2.288

Interpretação da cena: **1× bus, 4× person, 1× stop sign**, em 1.465 ms de inferência completa.

Contraste: características projetadas à mão × aprendidas

Para mostrar o que a área ganhou nos últimos vinte anos, o mesmo problema foi resolvido com **HOG + SVM** (Dalal & Triggs, 2005), o detector clássico que ainda acompanha o OpenCV.

	HOG + SVM (2005)	YOLOv8N (2023)
Descritor	escrito à mão	aprendido dos dados
Classes que sabe procurar	1 (pedestres)	80
Detecções nesta imagem	3 pessoas	4 pessoas + ônibus + placa
Tempo	739 ms	1.465 ms

AREA 3 — Visão Computacional: da imagem para a descrição da cena

PASSO 1 — Aquisição
810x1080 pixels



Clássico: HOG + SVM
3 pessoa(s), 1 classe



YOLOv8n: 6 objetos
1x bus, 4x person, 1x stop sign



A imagem deixa claro o que os números escondem: as caixas do HOG são frouxas e mal posicionadas, e o detector **não tem sequer vocabulário** para dizer “ônibus”.

BALANÇO DA ÁREA 3

- **Entrada:** imagem matricial 810×1080×3 = **2.624.400** bytes.
- **Saída:** descrição simbólica — 6 registros, **974 bytes** de JSON.
- Uma redução de **2.694×** no volume de dados: a área troca pixels por **significado**.
- A imagem anotada é apenas para o humano ver. O produto real é a **lista de objetos** — é ela que alimenta o freio automático, o catálogo, o alarme.

4

Visualização Computacional

VTK / PyVista · Matplotlib · duas frentes: SciVis e InfoVis

dados medidos ou abstratos → imagem para análise

Técnicas da CG para representar dado/informação: representações gráficas de dados, numéricos ou não. Objetivos: facilitar o entendimento de fenômenos complexos e a exploração de diferentes cenários. Síntese para gerar as representações visuais, análise (pelo usuário) para extrair informações.

aula02/intro_computacao_visual.pdf, slide 47

O mesmo material (slide 50) separa a área em duas frentes, e ambas foram executadas: em **SciVis** a geometria do modelo é *determinada pelo domínio*; em **InfoVis** ela é *atribuída pelo projetista*.

Parte A · Visualização Científica

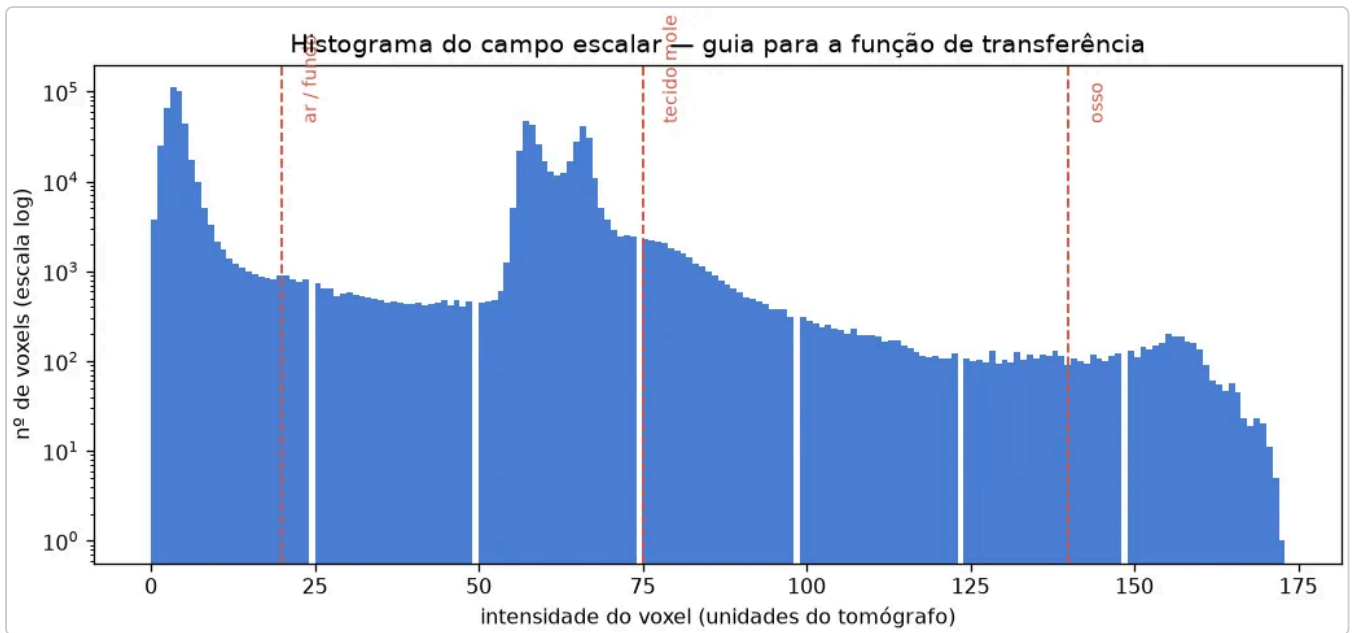
REPOSITÓRIOS	github.com/Kitware/VTK · github.com/pyvista/pyvista
DADO	tomografia computadorizada de joelho, de github.com/pyvista/vtk-data
PROGRAMA	<code>04_visualizacao_computacional/visualizacao_vtk.py</code>

A escolha vem direto do slide 54, que exibe um “*Rendering Volumétrico Direto: ray casting no Visualization Toolkit*”.

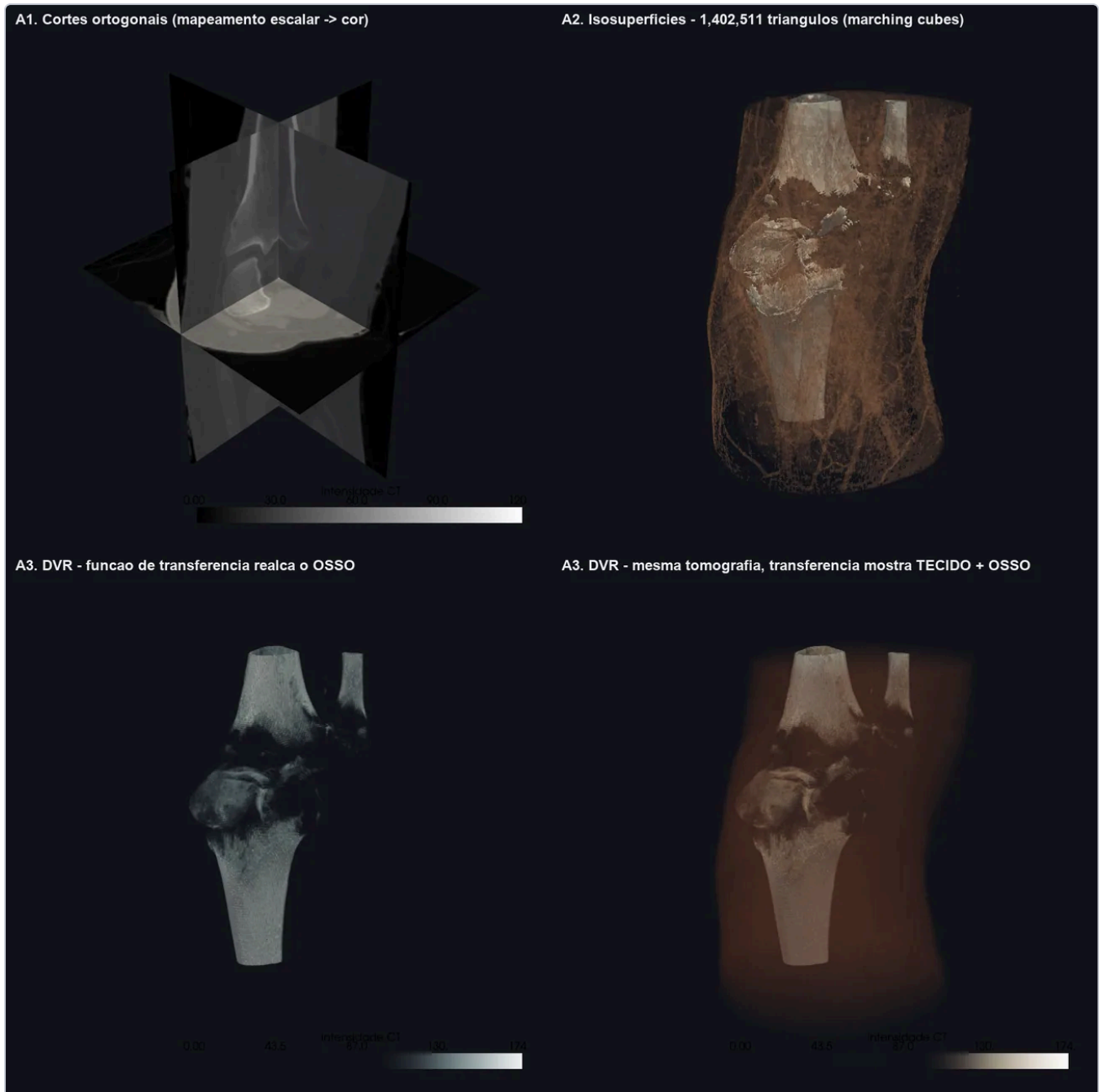
O DADO DE ENTRADA

```
dimensões da grade .....: 208 x 248 x 201
total de voxels .....: 10.368.384
espaçamento (mm) .....: (0.723, 0.723, 1.0)
faixa de intensidade .....: 0 a 174
memória do campo .....: 9,9 MB
```

Não é uma imagem e não é uma malha: é uma função $f(x,y,z)$ MEDIDA.
Não existe superfície nenhuma no arquivo – só valores.



O histograma do campo é o que orienta todas as decisões seguintes: é ele que diz onde termina o ar, onde está o tecido mole e a partir de qual valor começa o osso.



Três técnicas sobre o **mesmo** dado. Note o custo da isosuperfície: **1,4 milhão de triângulos** para representar aquilo que o DVR mostra sem gerar um único polígono.

1. **Cortes ortogonais** — o mapeamento mais direto possível: intensidade → cor, sem construir geometria alguma.
2. **Isosuperfície** (*marching cubes*) — extrai geometria do campo e devolve o problema ao pipeline da Área 1: isovalor 62 (tecido mole) → 1.226.157 triângulos; isovalor 108 (osso) → 176.354 triângulos.
3. **Rendering Volumétrico Direto** (*ray casting*) — o raio de cada pixel atravessa o volume acumulando cor e opacidade. Nenhuma superfície é construída.

O QUE DECIDE O QUE APARECE NO DVR: PONTOS DE CONTROLE EM [0, 34, 69, 104, 139, 174]

FUNÇÃO DE TRANSFERÊNCIA	OPACIDADES	RESULTADO
realce do osso	[0, 0, 0, 0.05, 0.60, 0.95]	tecido mole invisível, osso sólido
osso + tecido	[0, 0, 0.01, 0.06, 0.70, 0.98]	pele translúcida, osso visível por dentro

Os dois últimos quadros da montagem são **exatamente a mesma tomografia, a mesma câmera e o mesmo algoritmo** — muda só a função de transferência. É a demonstração mais limpa de que, em Visualização, **a decisão de projeto é o mapeamento, não a forma.**

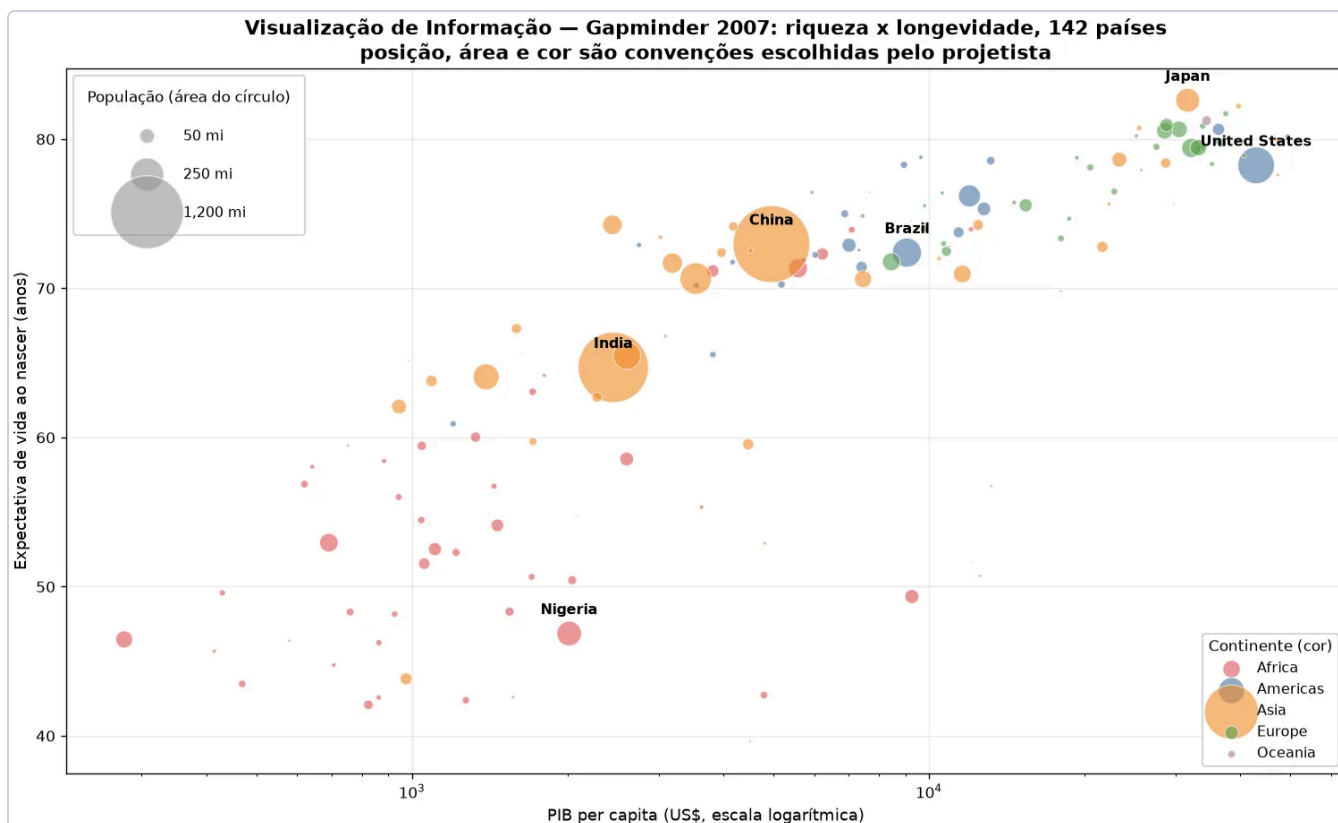
Parte B · Visualização de Informação

REPOSITÓRIOS	github.com/matplotlib/matplotlib · dados de github.com/plotly/datasets
DADO	<code>gapminder2007.csv</code> — 142 países × 5 variáveis, 19 KB

Reprodução do gráfico do **Gapminder**, citado no slide 62. A tabela de entrada não tem geometria nenhuma associada: **um país não tem posição (x, y) natural — ela precisa ser inventada.**

MAPEAMENTO VISUAL ESCOLHIDO PELO PROJETISTA

CANAL VISUAL	VARIÁVEL
posição X	PIB per capita (US\$, escala logarítmica)
posição Y	expectativa de vida (anos)
área do círculo	população
cor	continente (categórica)



Correlação entre log(PIB) e expectativa de vida: **0,809**. A expectativa de vida varia de 39,6 a 82,6 anos — **43 anos** entre o pior e o melhor caso.

LEITURA POSSIBILITADA PELO GRÁFICO

CONTINENTE	N	VIDA MÉDIA	PIB MEDIANO
Africa	52	54,8 anos	US\$ 1.452
Americas	25	73,6 anos	US\$ 8.948
Asia	33	70,7 anos	US\$ 4.471
Europe	30	77,6 anos	US\$ 28.054
Oceania	2	80,7 anos	US\$ 29.810

BALANÇO DA ÁREA 4

- **Entrada:** dados que não são imagem nem modelo geométrico de um objeto — um campo escalar medido (**10.368.384** voxels) e uma tabela de indicadores (**142** linhas).
- **Saída:** imagem cujo propósito é **análise**, não realismo. A pergunta não é “ficou bonito?”, e sim “ficou legível?”.
- SciVis × InfoVis: a forma do joelho **veio do tomógrafo**; a posição do Brasil no gráfico **foi escolhida por quem desenhou o gráfico**.
- Ler o joelho é intuitivo; ler o Gapminder exige saber que o eixo X é logarítmico e que é a *área* — não o raio — que codifica a população.

Conclusão: o que separa e o que une

O que separa é o par (entrada, saída), demonstrado numericamente pelas execuções.

TODAS AS MEDIDAS VÊM DE RELATORIO/LOG_EXECUCAO.TXT

ÁREA	ENTRADA MEDIDA	SAÍDA MEDIDA	TRANSFORMAÇÃO
1 Síntese	19.895 vértices / 37.986 triângulos (466 KB)	imagem 900×900 (810.000 px)	expande geometria em pixels
2 Processamento	imagem 512×480×3	imagem 512×480	preserva o domínio
3 Visão	imagem 810×1080×3 (2,6 MB)	6 objetos (974 bytes)	comprime 2.694× em significado
4 Visualização	10.368.384 voxels (9,9 MB) / 142 linhas (19 KB)	imagem 900×900 para análise	traduz dado em forma visível

O que une apareceu sozinho durante a execução, sem ter sido forçado:

- A **Área 3 usa a Área 2**: todo o Passo 2 do YOLO — *letterbox*, conversão de espaço de cor, normalização — é Processamento de Imagens puro.
- A **Área 3 redescobriu a Área 2**: os mapas de características do bloco 0 do YOLO são detectores de borda, visualmente equivalentes ao Sobel da Área 2. A diferença é que um foi escrito por uma pessoa e o outro foi aprendido de dados.
- A **Área 4 usa a Área 1**: a isosuperfície por *marching cubes* produz 1,4 milhão de triângulos que são rasterizados exatamente pelo pipeline da Área 1. O DVR é a alternativa que dispensa esse passo.
- As **Áreas 1 e 4 compartilham o motor e divergem no objetivo**: as duas fazem `dados → imagem`, mas uma persegue realismo e a outra, legibilidade.

O ponto do slide 66 — *identidade própria, focos distintos, técnicas compartilhadas* — não é uma ressalva acadêmica. É o que se observa assim que os quatro programas são executados lado a lado.

Estrutura da entrega

```
AC1/
├─ README.md           relatório em Markdown
├─ requirements.txt     versões exatas usadas na execução
├─ executar_tudo.py     roda as quatro áreas em sequência
├─ assets/             dados de entrada (repositórios públicos)
├─ 00_comparativo/     diagrama e painel das quatro áreas
├─ 01_sintese_imagens/  ModernGL / OpenGL 3.3 Core
├─ 02_processamento_imagens/  OpenCV
├─ 03_visao_computacional/  Ultralytics YOLOv8n + HOG/SVM
├─ 04_visualizacao_computacional/  VTK/PyVista + Matplotlib
├─ relatorio/
│   └─ log_execucao.txt  saída de console das cinco execuções
│   └─ relatorio.html    este documento
```

Repositórios públicos executados

ÁREA	PROJETO	ENDEREÇO
1	ModernGL	github.com/moderngl/moderngl
2	OpenCV	github.com/opencv/opencv
3	Ultralytics YOLO	github.com/ultralytics/ultralytics
3	Ultralytics assets	github.com/ultralytics/assets
4	VTK — Visualization Toolkit	github.com/Kitware/VTK
4	PyVista	github.com/pyvista/pyvista
4	vtk-data (tomografia)	github.com/pyvista/vtk-data
4	Matplotlib	github.com/matplotlib/matplotlib
4	plotly/datasets (Gapminder)	github.com/plotly/datasets

Algoritmos citados

- Phong, B. T. (1975). *Illumination for computer generated pictures*.
- Otsu, N. (1979). *A threshold selection method from gray-level histograms*.
- Canny, J. (1986). *A computational approach to edge detection*.
- Lorensen, W. & Cline, H. (1987). *Marching cubes: a high resolution 3D surface construction algorithm*.
- Dalal, N. & Triggs, B. (2005). *Histograms of oriented gradients for human detection*.

- Redmon, J. et al. (2016). *You Only Look Once: unified, real-time object detection*.